Ayudantía 2

Lectura, limpieza y escritura de catálogos estelares

Física Computacional II – Ayudante: Martín Estay Valdés

Objetivos

- Escribir y leer archivos de texto y CSV en Python usando `with open(...)`, el módulo `csv` y `np.genfromtxt`.
- Manejar rutas y directorios con `pathlib`, creando las carpetas de salida desde el propio código.
- Organizar la lectura y el análisis de un catálogo dentro de una clase, con métodos que se llaman entre sí.
- Filtrar registros inválidos con máscaras booleanas, manteniendo la correspondencia entre columnas.
- Calcular propiedades físicas de forma vectorizada: flujo bolométrico a partir de la luminosidad y la distancia.
- Guardar los mismos datos en formatos NPZ, Pickle y JSON, verificar su integridad y comparar sus tamaños.

1 Teoría

Rutas y directorios

El módulo `pathlib` representa rutas como objetos. El operador `/` une carpetas y nombres de archivo, y funciona igual en Linux, macOS y Windows.

```
from pathlib import Path
ruta = Path("salidas") / "catalogo.npz"
ruta.parent.mkdir(parents=True, exist_ok=True) # crea "salidas/" si no existe
ruta.exists() # True o False
ruta.stat().st_size # tamaño del archivo en bytes
```

Con `exist_ok=True` no se produce un error si la carpeta ya existe, por lo que la línea puede repetirse en cada método que guarde archivos.

Archivos de texto y CSV

Un archivo se abre con `open(ruta, modo)`, donde el modo es `"r"` (leer), `"w"` (escribir, borra el contenido previo) o `"a"` (agregar al final). Para archivos binarios se agrega una `b`: `"rb"`, `"wb"`. Se usa siempre dentro de un bloque `with`, que cierra el archivo al terminar, aunque ocurra un error.

Un CSV guarda una tabla en texto: una fila por línea y las columnas separadas por comas. La primera línea suele contener los encabezados. Al leerlo con el módulo `csv` todo llega como `str`; en cambio, `np.genfromtxt` con `names=True` y `dtype=None` infiere el tipo de cada columna y devuelve un *arreglo estructurado*, cuyas columnas se acceden por nombre:

```
datos = np.genfromtxt(ruta, delimiter=",", names=True, dtype=None, encoding="utf-8")
datos.dtype.names # nombres de las columnas
datos["distancia_pc"] # columna como arreglo de NumPy
```

Los valores escritos como `nan` se leen como `np.nan`.

Clases

Una clase agrupa datos (atributos) y funciones que trabajan sobre ellos (métodos). El método `__init__` se ejecuta al crear una instancia, y `self` se refiere a la instancia misma. Un método puede llamar a otro con `self.metodo()`.

```
class Muestra:
    def __init__(self, valores):
        self.x = np.asarray(valores, dtype=float)
        self.normalizar() # se ejecuta al crear el objeto

    def normalizar(self):
        self.x = self.x / self.x.max()

m = Muestra([2, 4, 8])
m.x # array([0.25, 0.5 , 1. ])
```

Máscaras booleanas

Una comparación entre un arreglo y un número produce un arreglo de `True`/`False` del mismo largo. Al indexar con él se conservan solo los elementos marcados como `True`. Si se aplica *la misma* máscara a todas las columnas, cada estrella conserva sus datos.

Funciones útiles para construir máscaras: `np.isfinite` (falso para `nan` e `inf`), los operadores `&`, `|` y `~`, y `np.all(..., axis=0)`. Para detectar repetidos, `np.unique(a, return_index=True)` devuelve, además de los valores únicos, el índice de la *primera* aparición de cada uno.

Flujo bolométrico

Una estrella de luminosidad L emite energía en todas direcciones. A una distancia d esa energía se reparte sobre una esfera de área $4\pi d^2$, de modo que el flujo recibido es

$$F = \frac{L}{4\pi d^2}. \tag{1}$$

Para obtener F en $\mathrm{W\,m^{-2}}$ hay que convertir unidades: $L_\odot = 3.828 \times 10^{26}$ W y 1 pc $= 3.0857 \times 10^{16}$ m. Dos estrellas a la misma distancia difieren en flujo solo por su luminosidad; una estrella al doble de distancia se ve cuatro veces más débil.

Formatos de almacenamiento

- **NPY / NPZ**: binarios de NumPy. `np.save` guarda un arreglo; `np.savez` y `np.savez_compressed` guardan varios, cada uno con un nombre que luego funciona como clave. Conservan el `dtype` y la forma.
- **Pickle**: serializa casi cualquier objeto de Python (diccionarios, listas, arreglos). Solo sirve dentro de Python y **nunca** debe abrirse un Pickle de origen desconocido, porque cargarlo puede ejecutar código.
- **JSON**: texto legible y compatible con otros lenguajes. No admite arreglos de NumPy directamente; hay que convertirlos con `.tolist()`.

Para cargar un NPZ se recomienda `allow_pickle=False`. Esto funciona solo si los textos están guardados como arreglos de tipo `U` (por ejemplo `<U10`); un arreglo de tipo `object` requiere Pickle para leerse.

Para verificar que un archivo recuperado coincide con el original se usa `np.array_equal` (igualdad exacta, adecuada para textos) y `np.allclose` (igualdad dentro de una tolerancia, adecuada para números de punto flotante).

2 Ejercicios

Ejercicio 1

Genere un catálogo sintético de $N = 15$ estrellas e incluya en él algunos errores típicos, para luego practicar su limpieza. Use `rng = np.random.default_rng(2026)`, definido una sola vez.
Comandos: `rng.uniform`, `np.array`, `np.nan`, `Path.mkdir`, `open`, `csv.writer`, `writerow`, `Path.read_text`, `splitlines`, `ax.scatter`, `ax.set_yscale`.

(a) Cree los identificadores `EST-001`, ..., `EST-015` y los nombres `Estrella_A`, ..., `Estrella_O`. Luego genere, en este orden, temperaturas uniformes entre 3000 y 10000 K, luminosidades uniformes entre 0.5 y 50 $L_\odot$ y distancias uniformes entre 10 y 200 pc.

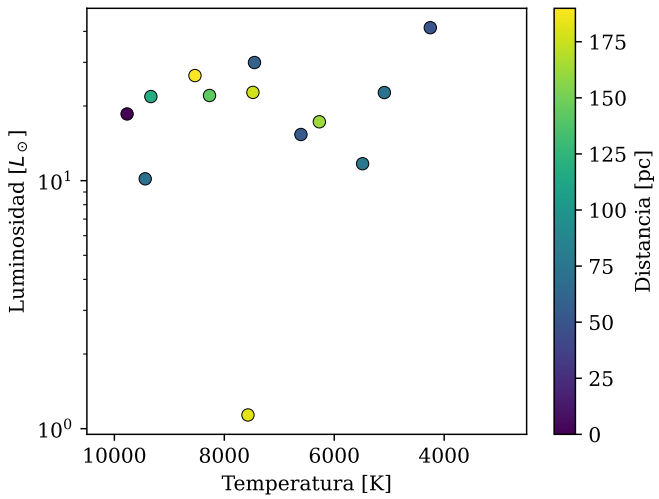
(b) Introduzca los siguientes errores: temperatura `nan` en el índice 3, luminosidad -2 en el índice 7, distancia 0 en el índice 10, y copie el identificador del índice 5 en el índice 12.

(c) Guarde el catálogo en `datos/catalogo_sucio.csv` con los encabezados `id`, `nombre`, `temperatura_K`, `luminosidad_Lsol` y `distancia_pc`, usando cuatro decimales. La carpeta `datos/` debe crearse desde el código. Imprima las primeras cinco líneas del archivo.

(d) Grafique la luminosidad en función de la temperatura, con el eje de temperatura invertido, el eje de luminosidad en escala logarítmica y el color de cada punto según su distancia. ¿Qué registros no aparecen en el gráfico y por qué?

Salida esperada:

```
id,nombre,temperatura_K,luminosidad_Lsol,distancia_pc
EST-001,Estrella_A,4252.5437,41.3818,49.7891
EST-002,Estrella_B,7479.3922,22.6948,176.1786
EST-003,Estrella_C,6270.8788,17.2712,161.5178
EST-004,Estrella_D,nan,14.2560,125.2748
```



Ejercicio 2

Cree una clase `Catalogo` cuyo constructor reciba la ruta de un CSV con el formato del Ejercicio 1, lo lea y guarde cada columna como un atributo independiente (`self.id`, `self.nombre`, `self.temperatura`, etc.).
Comandos: `np.genfromtxt`, `np.isfinite`, `np.vstack`, `np.all`, `np.unique`, `np.zeros`, `np.argmax`, `np.argmin`, `ax.loglog`.

(a) Implemente un método `limpiar()` que elimine los registros con valores no finitos o no positivos en las columnas numéricas y que, para identificadores repetidos, conserve solo la primera aparición. El método debe aplicarse a todas las columnas e informar cuántos registros eliminó. Llámelo desde `__init__`.

(b) Llame `limpiar()` una segunda vez sobre el mismo objeto. ¿Cambia el número de estrellas? ¿Por qué es importante que así sea?

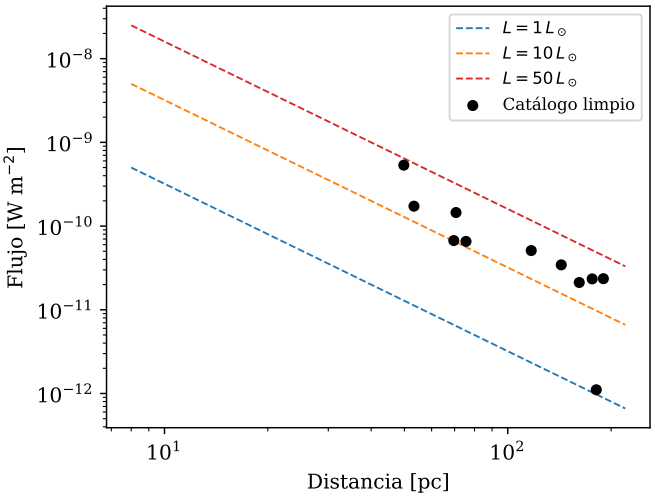
(c) Implemente un método `flujo()` que devuelva el flujo bolométrico de cada estrella en $\mathrm{W\,m^{-2}}$, sin usar ciclos.

(d) Implemente un método `resumen()` que imprima el número de estrellas, la distancia media, el flujo máximo junto al nombre de la estrella que lo presenta, y la estrella más fría.

(e) Grafique el flujo en función de la distancia en escala log-log, junto con las curvas $F(d)$ para $L = 1$, 10 y 50 $L_\odot$. ¿Qué pendiente tienen esas curvas? ¿Entre qué valores de luminosidad se encuentran las estrellas del catálogo?

Salida esperada:

```
Registros eliminados: 4 de 15
Registros eliminados: 0 de 11
Número de estrellas    : 11
Distancia media        : 117.1 pc
Flujo máximo           : 5.341e-10 W/m^2 (Estrella_A)
Estrella más fría      : Estrella_A (4253 K)
```



Ejercicio 3

Agregue a la clase `Catalogo` métodos para guardar el catálogo limpio. Cada método debe recibir la ruta de destino y crear por sí mismo la carpeta, de modo que funcione aunque no se haya llamado antes a ningún otro método. Use la carpeta `salidas/`.
Comandos: `np.savez`, `np.savez_compressed`, `np.load`, `pickle.dump`, `pickle.load`, `json.dump`, `json.load`, `.tolist()`, `np.array_equal`, `np.allclose`, `Path.stat`, `ax.bar`.

(a) `guardar_npz(ruta)`: guarda todas las columnas con `np.savez_compressed`, usando el nombre de cada atributo como clave.

(b) `guardar_pickle(ruta)` y `guardar_json(ruta)`: guardan un diccionario con todas las columnas.

(c) `guardar_brillantes(ruta, F_min)`: guarda en CSV solo las estrellas con flujo mayor que `F_min` y devuelve cuántas son. Úselo con $F_{\min} = 10^{-10}$ W m^{-2}.

(d) Lea los archivos NPZ (con `allow_pickle=False`), Pickle y JSON, y compruebe que los identificadores y las temperaturas coinciden con el catálogo limpio. Imprima también el `dtype` de los nombres recuperados del NPZ. ¿Qué ocurre si los nombres se guardan como un arreglo de tipo `object`?

(e) Guarde además una versión con `np.savez` (sin comprimir) y construya una tabla y un gráfico de barras con el tamaño en bytes de los cuatro archivos. ¿Por qué el JSON, que es texto, resulta más pequeño que los formatos binarios en este caso? ¿Esperaría lo mismo con un catálogo de un millón de estrellas?

Salida esperada:

```
Estrellas con F > 1e-10 W/m^2: 3
NPZ  ids: True | T: True | dtype nombre: <U10
PKL  ids: True | T: True
JSON ids: True | T: True
Formato              Bytes
NPZ sin comprimir     2272
NPZ comprimido        1375
Pickle                1401
JSON                   919
```

